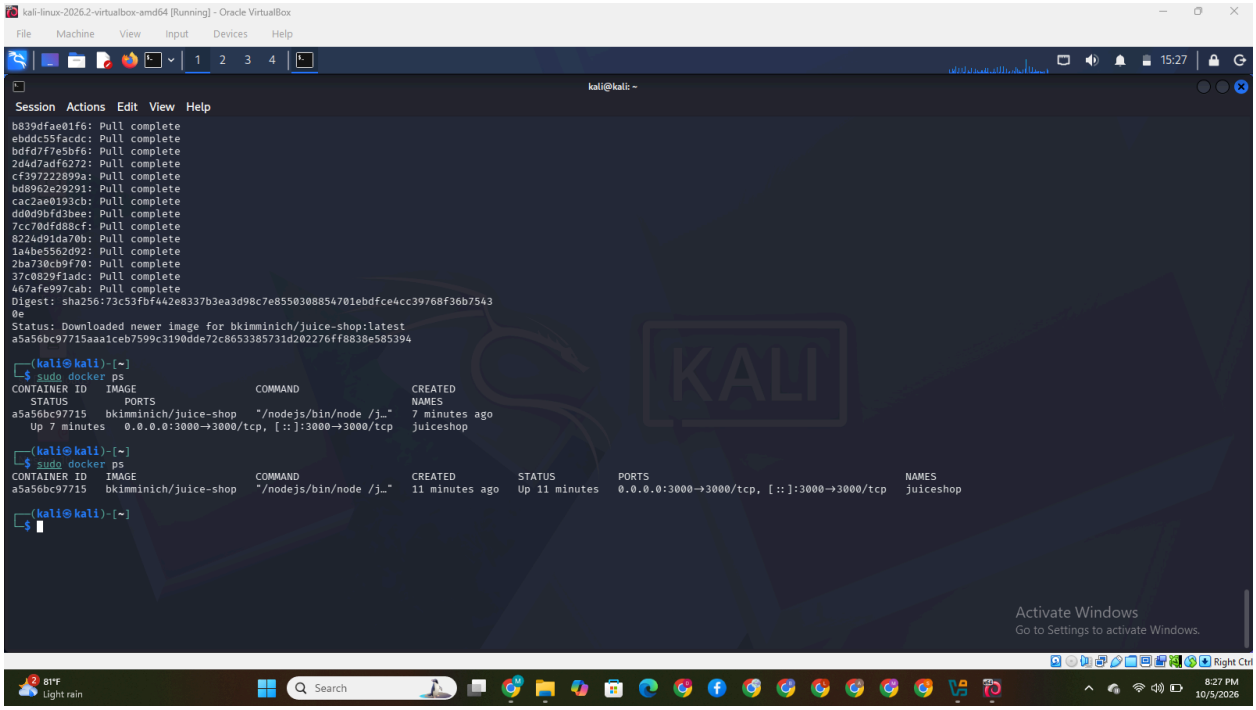
Attack Surface Mapping & Reconnaissance Report: OWASP Juice Shop

Prepared By: Mercy Itunuoluwa Olalere
Target Application: OWASP Juice Shop (v17.1.1)
Scope: Reconnaissance, Technology Fingerprinting & Attack Surface Mapping (No Exploitation)

1. Environment Setup Documentation

The target application was deployed in a local virtual environment to model an isolated testing infrastructure.

- **Operating System / Environment:** Linux / Ubuntu
- **Deployment Method:** Local deployment on Port 3000 via Node.js runtime.
- **Network & Host Binding:** http://localhost:3000 (127.0.0.1).
- **Verification:** The web interface rendered successfully, responding to client HTTP GET requests.



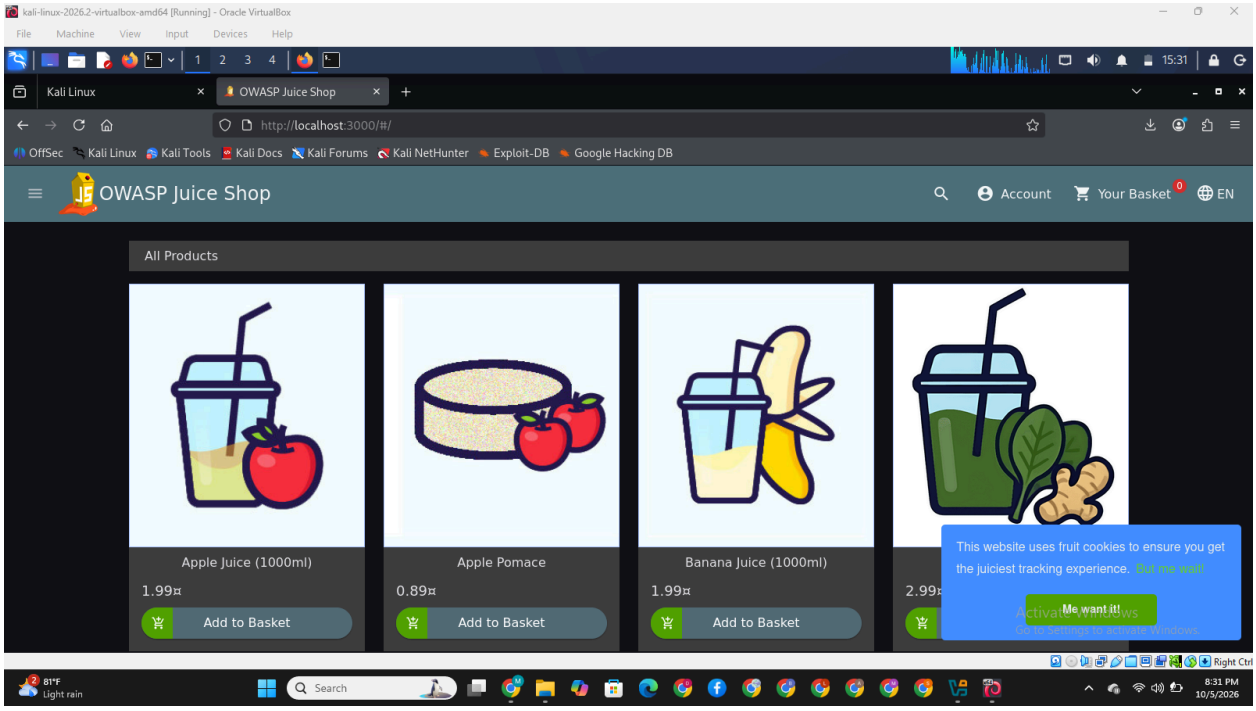
Terminal window showing 'npm start' or process listening on port 3000, and Nmap scan output

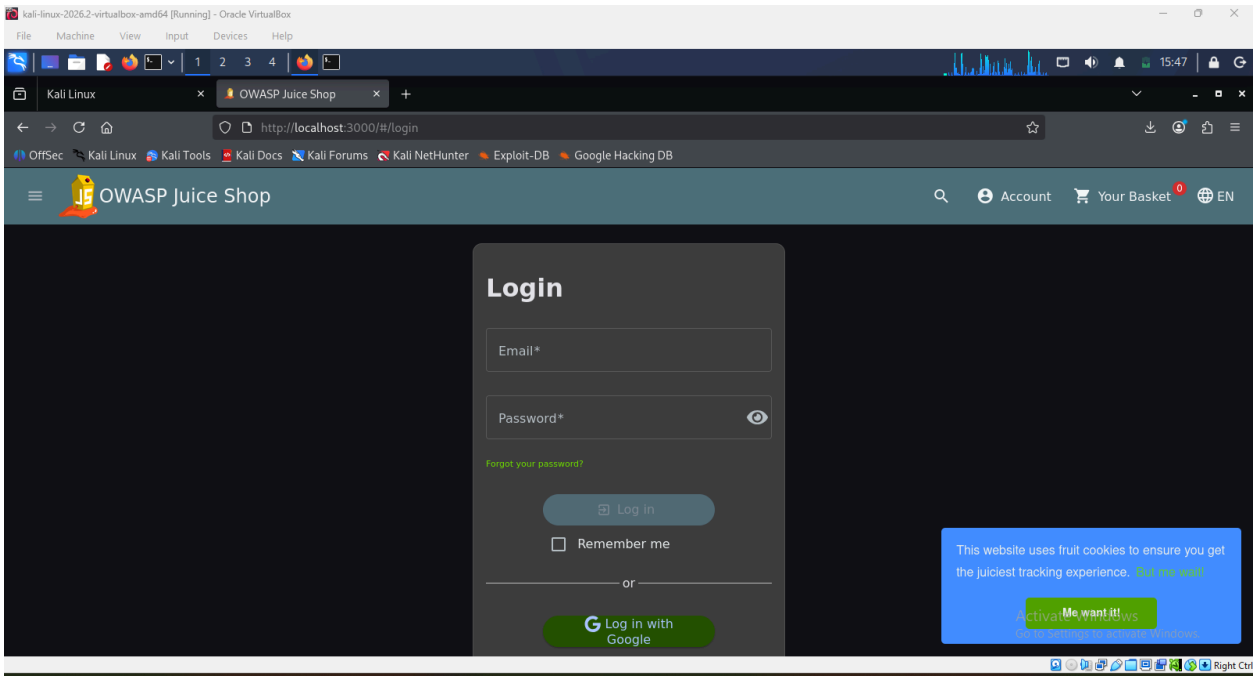
2. Reconnaissance Report

2.1 Web Application & Route Discovery

Manual traversal and client-side analysis revealed key functional interfaces:

- **Public Pages:** Main catalog (/#/search), Customer Feedback, About Us.
- **Authentication Pages:** User Login (/#/login), User Registration (/#/register), Forgot Password (/#/forgot-password).
- **User-Restricted Pages:** User Profile, Shopping Basket (/#/basket), Order History.
- **Admin Functions:** Administrative Dashboard (/#/administration) identified via frontend JavaScript route inspection.



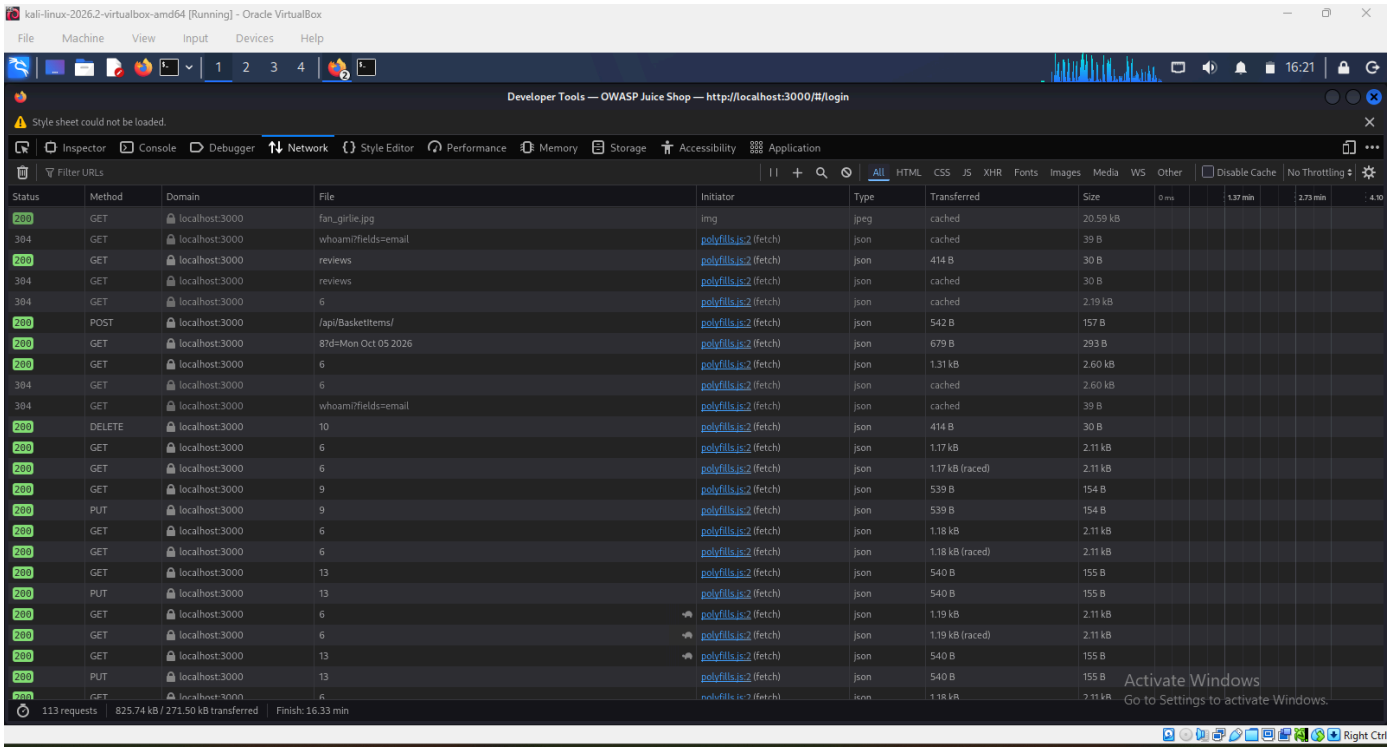


Registration interface screenshot

2.2 API & Endpoint Mapping

Intercepted network traffic logs (captured via HAR archive) identified key backend routes and HTTP methods:

- `GET /api/SecurityQuestions/` – Fetches registration security questions.
- `POST /api/Users/` – Processes account creation payloads.
- `POST /api/SecurityAnswers/` – Registers account recovery answers.
- `POST /rest/user/login` – Authenticates user credentials and returns JSON Web Tokens (JWT).
- `GET /rest/user/whoami` – Checks active token validity.
- `GET /rest/basket/{id}` – Retrieves shopping cart contents by numeric ID.
- `GET /rest/admin/application-configuration` – Exposes server settings without requiring authentication.



Browser Developer Tools Network tab or Burp Suite HTTP history showing API requests

2.3 Technology Fingerprinting

System inspection, HTTP response headers, and source inspection confirmed the full application stack:

- **Frontend Framework:** Angular Single Page Application (SPA) (`main.js`, `runtime.js`).

- **Backend Framework:** Node.js using Express web framework (X-Powered-By: Express).
- **Database Technology:** SQLite with Sequelize ORM.
- **Authentication Mechanism:** JSON Web Token (JWT) Bearer tokens.

3. Attack Surface Inventory

Asset / Endpoint	Type	Access	Method	Technology	Security Relevance	Priority
http://localhost:3000/	Web	Public	GET	Angular SPA	Entry point to client-side bundles and unlinked JavaScript routes.	Low
/rest/admin/application-configuration	API	Public	GET	Node.js / Express	Sensitive Data Leakage: Exposes OAuth parameters, chatbot configurations, and system defaults without auth.	High
/rest/user/login	API	Public	POST	Express / JWT	Handles primary authentication. Contains hashed password strings inside returned JWT payloads.	High
/rest/basket/{id}	API	Authenticated	GET	Node.js / Express	IDOR Risk: Basket references use simple sequential integers (basket/6), enabling object tampering.	High

/api/Users/	API	Public	POST	Express / SQLite	Registration vector for SQL Injection or Broken Object Level Authorization (BOLA) during user creation.	Medium
/api/SecurityQuestions/	API	Public	GET	Express / SQLite	Enumeration vector for security questions.	Low

4. Attack Surface Diagram

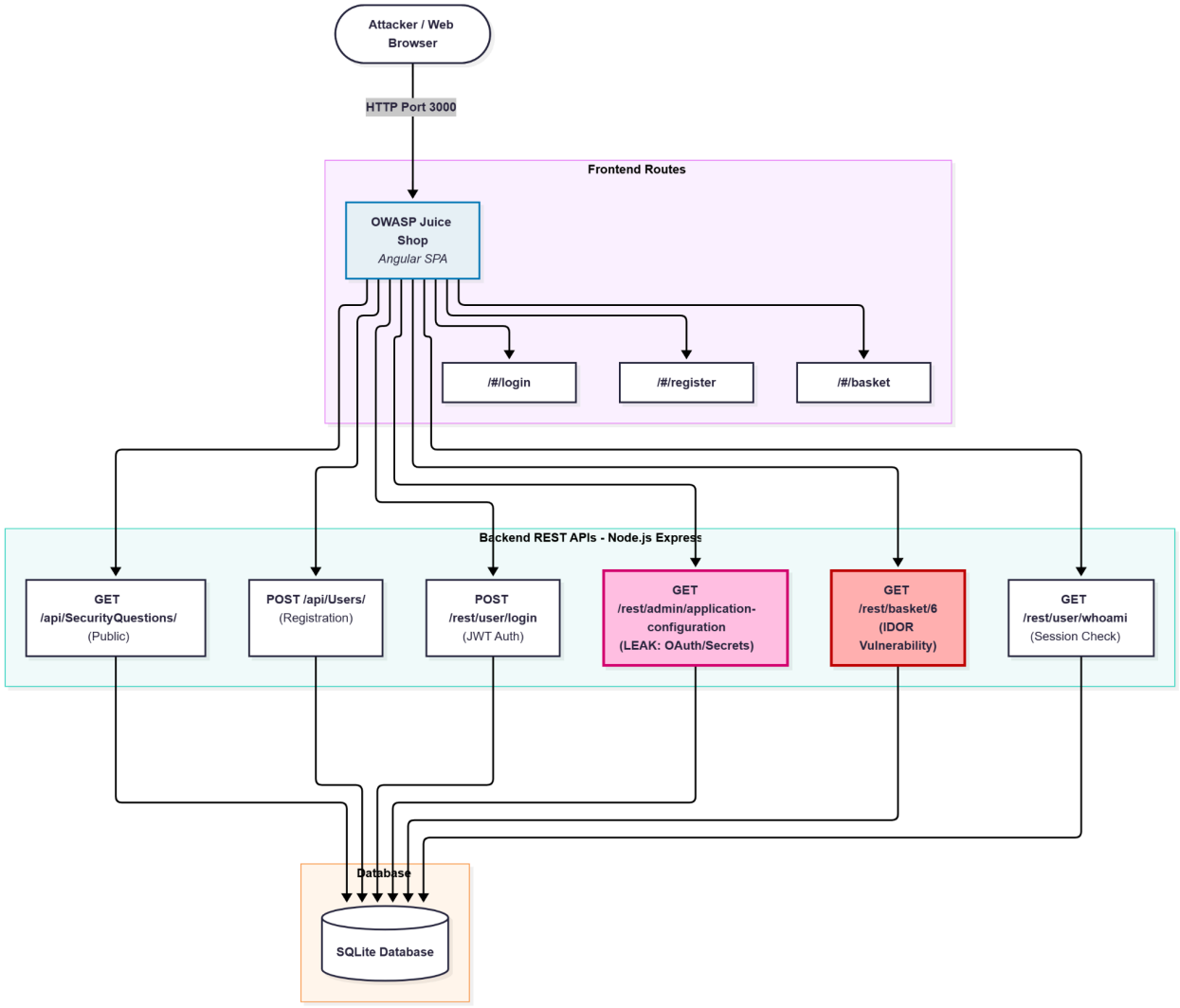


Figure 1: OWASP Juice Shop Attack Surface Architecture Diagram

This diagram illustrates the mapped attack surface of the OWASP Juice Shop application, tracing user traffic through client-side Angular routes to backend Express REST APIs and the underlying SQLite database. High-priority exposure points, such as unauthenticated configuration disclosure (/rest/admin/application-configuration) and IDOR vulnerabilities (/rest/basket/{id}), are highlighted in bold red/pink to indicate primary targets for subsequent security testing.

5. Prioritized Testing Plan

Follow-up security assessments should focus on identified risk vectors in the following order:

High-Priority Areas

1. **Unauthenticated Configuration Access (/rest/admin/application-configuration):**
    - *Reasoning:* Exposes internal routes, third-party credentials, and system settings without authentication.
2. **Insecure Direct Object References in Basket Operations (/rest/basket/{id}):**
    - *Reasoning:* Sequential ID usage allows attackers to access or modify other users' shopping carts by incrementing the ID parameter.
3. **Authentication & JWT Token Security (/rest/user/login):**
    - *Reasoning:* Inspect JWT claims for excessive disclosures (such as password hashes) and weak signature verification.

Medium-Priority Areas

1. **User Registration & Input Fields (/api/Users/):**
    - *Reasoning:* Check for input sanitization flaws, potential SQL/NoSQL injections, or mass-assignment vulnerabilities.

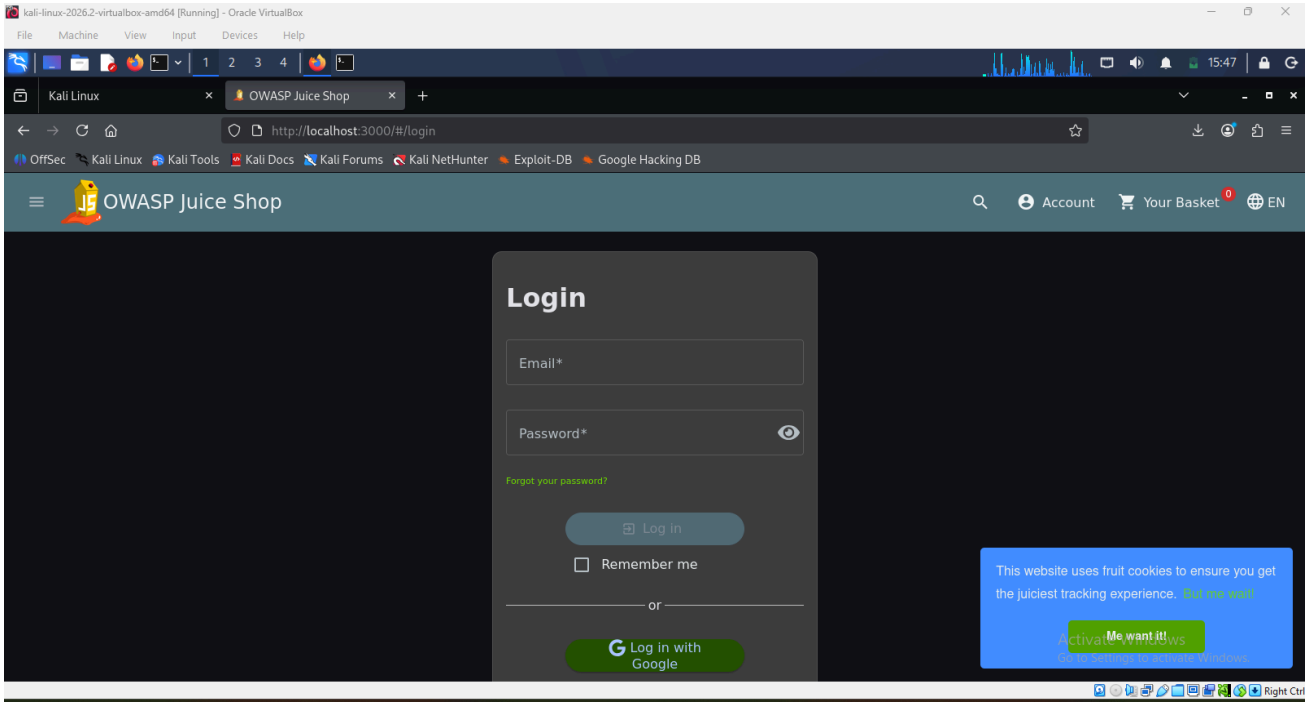
Low-Priority Areas

1. **Public Static Endpoints (/api/SecurityQuestions/):**
    - *Reasoning:* Low impact; primarily returns static informational data.

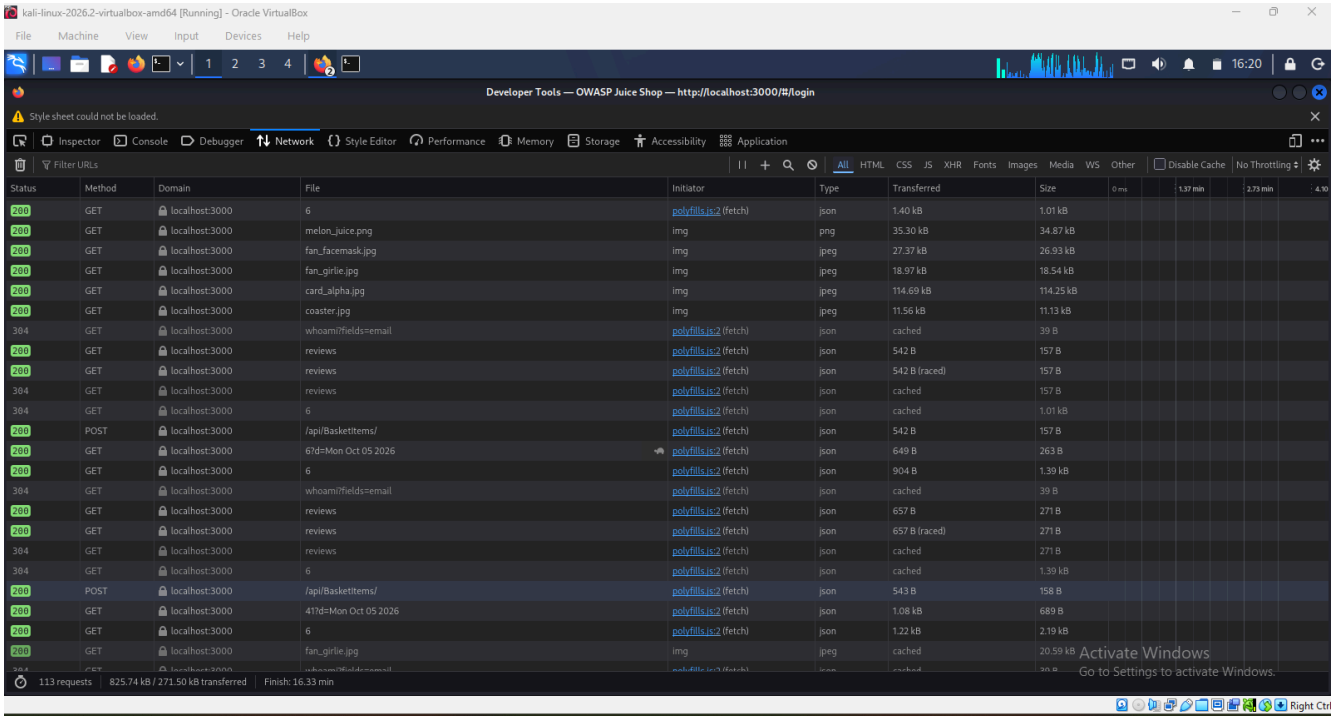
6. Evidence

- **Captured HAR Logs:** Network traffic trace file (localhost_Archive [26-10-05 16-22-38].har).
- **Port Scan Output:** Nmap scan result verifying active service parameters on port 3000.

- **Captured UI Screenshots:** User Registration, Login flow, and Developer Console network logs.



Registration Form UI



JWT response payload inside browser Developer Tools